SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

*I(**Sandesh S/192521078**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandesh S

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



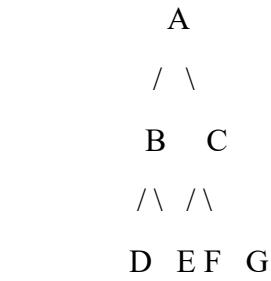
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1	2	
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1

2

3

4

5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

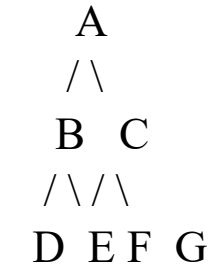
RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.

Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1: Breadth-First Search (BFS)

Step 1: Understand the given tree



BFS visits nodes level by level, from left to right.

Step 2: Start with the root node

- Start node: A
- Queue: [A]

- Visited: []

Step 3: Perform the dry run

In BFS, remove the first node from the queue, visit it, and add its children to the end of the queue.

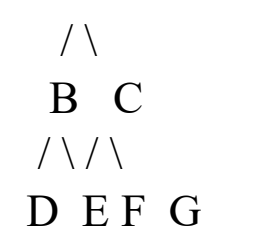
Iteration	Queue before	Node visited	Queue after
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Step 4: Final answers

1. BFS traversal: A → B → C → D → E → F → G
2. Data structure used: Queue (FIFO).
3. Queue after Iteration 3: [D, E, F, G]
4. Queue after Iteration 7: Empty.

Question 2: Depth-First Search (DFS)

Step 1: Understand the given tree



DFS explores one branch completely before moving to the next branch.

Step 2: Start with the root node

- Start node: A
- Stack: [A]
- Visited: []

DFS uses a stack (LIFO). Here, the top of the stack is shown on the left. Push the right child first so the left child is visited first.

Step 3: Perform the dry run

Iteration	Stack before	Node visited	Stack after
1	A	A	B, C
2	B, C	B	D, E, C
3	D, E, C	D	E, C
4	E, C	E	C
5	C	C	F, G
6	F, G	F	G
7	G	G	Empty

Step 4: Final answers

1. DFS traversal: A → B → D → E → C → F → G 2.
   Data structure used: Stack (LIFO).
   3. Stack after Iteration 3: [E, C] 4.
   Stack after Iteration 7: Empty.
   5. Difference: BFS explores level by level, whereas DFS explores one branch deeply before returning.

Question 3: 8-Puzzle Using BFS

Step 1: Write the initial state

1 3 6
5 0 2
4 7 8

Here, 0 represents the empty space.

Step 2: Write the goal state

1 2 3
4 5 6
7 8 0

The aim is to move the empty space until the puzzle reaches the goal state.

Step 3: Understand how BFS works

1. Put the initial state into the queue.
2. Remove the first state from the queue.
3. Move the empty space Up, Down, Left, or Right wherever a move is valid.
4. Add new, unvisited states to the queue.
5. Repeat until the goal state is found.

For the dry run below, the move order is Up → Down → Left → Right. We do not add states that have already been discovered.

Step 4: First five iterations

To make the queue easier to understand, use these labels:

- S0 = Initial state
- S1 = Move Up from S0
- S2 = Move Down from S0
- S3 = Move Left from S0
- S4 = Move Right from S0
- S5, S6 = New states generated while expanding S1
- S7, S8 = New states generated while expanding S2
- S9 = New state generated while expanding S3
- S10, S11 = New states generated while expanding S4

Iteration	Current state expanded	Queue before expansion	New states added	Queue after expansion
1	S0	S0	S1, S2, S3, S4	S1, S2, S3, S4
2	S1	S1, S2, S3, S4	S5, S6	S2, S3, S4, S5, S6
3	S2	S2, S3, S4, S5, S6	S7, S8	S3, S4, S5, S6, S7, S8
4	S3	S3, S4, S5, S6, S7, S8	S9	S4, S5, S6, S7, S8, S9
5	S4	S4, S5, S6, S7, S8, S9	S10, S11	S5, S6, S7, S8, S9, S10, S11

The table shows the first five expansions, not the entire search. The number of states generated later depends on the move order and duplicate-state handling.

Step 5: Write the complete solution path

One shortest solution reaches the goal in 8 moves.

1
3
6
5

2
4
7
8

Initial state
Move 1: Right
1
3
6
5
2
4
7
8

Step 1 Move 2: Up
1
3
5
2
6
4
7
8

Step 2 Move 3: Left
1
3
5

2
6
4
7
8

Step 3 Move 4: Down

1
2
3
5
6
4
7
8

Step 4 Move 5: Left

1
2
3
5
6
4
7
8

Step 5 Move 6: Down

1
2
3
4
5
6
7

8

Step 6 Move 7: Right
1
2
3
4
5
6
7
8

Move 8: Right — Goal reached
1
2
3
4
5
6
7
8
Goal state
Puzzle solved in 8 moves.